CFGS DAW · UT1 · DESPLIEGUE DE APLICACIONES WEB

Control de versiones PHP Words

Memoria de la actividad: repositorio local, ramas, fusiones y etiquetas

Alumno/a: #Nombre ficticio Apellido ficticio

Grupo: #2.º CFGS DAW (dato ficticio)

Centro: IES Ramón Arcas Meca

Fecha: 19/10/2026

Proyecto: `/home/xander/Escritorio/repo/php-words`

> **Nota sobre datos:** los datos personales ficticios llevan #. Esta memoria documenta el estado real del proyecto y sus commits. Los espacios de capturas son marcadores: no son capturas ni evidencias simuladas.

Índice

1. Misión 1. El punto de partida
2. Misión 2. Mirar antes de guardar
3. Misión 3. Documentación en su propia rama
4. Misión 4. Volver a main e incorporar la documentación
5. Misión 5. Tres ideas en paralelo
6. Misión 6. Fusionar, etiquetar y resolver
7. Misión 7. Etiquetas y limpieza
8. Reflexión final

Alcance y evidencias

El repositorio se creó localmente, sin remoto. El estado final tiene una sola rama (`main`), las etiquetas `v0.1.0`, `v0.2.0` y `v0.3.0`, 33 palabras y los dos filtros funcionando juntos.

> Las capturas requeridas por el guion deben hacerse en el escritorio del alumno/a y mostrar la pantalla completa, fecha/hora y ruta. Se señalan en cada misión con su código para sustituir estos recuadros por las capturas reales antes de entregar.

Misión 1 · El punto de partida

Se creó la aplicación estática PHP Words con un diseño adaptable, descripciones en español y 30 palabras iniciales. Se añadieron los seis archivos requeridos; `notas.txt` se creó aparte y se dejó intencionadamente fuera de Git.

Prompt de la aplicación

> # Reconstrucción del prompt para esta memoria: no se conservó una transcripción literal del prompt enviado. No debe presentarse como captura o cita original.

```
Crea una aplicación web estática llamada PHP Words para consultar palabras reservadas de PHP.
Genera estos seis archivos: index.html, README.md, css/estilos.css, js/config.js,
js/palabras.js y js/app.js. Respeta el contrato técnico completo indicado abajo. Decisiones de
diseño: usa una paleta azul marino y azul claro; redacta las descripciones en español, breves y
didácticas; incluye 30 palabras distribuidas entre las categorías permitidas. Diseña una
interfaz adaptable, clara y accesible. No generes notas.txt: lo crearé manualmente. En README
deja la sección Cómo ejecutar con el texto Pendiente de documentar.
```

Contrato técnico incluido en el prompt

```
Sistema de ficheros del proyecto
php-words/
├── index.html
├── README.md
├── notas.txt (la creo a mano en la Misión 1)
├── css/
│   └── estilos.css
└── js/
    ├── config.js
    ├── palabras.js
    └── app.js

Reglas fijas
index.html carga al final del body, en este orden y una línea por script: js/config.js,
js/palabras.js y js/app.js. Sin módulos, librerías externas ni fetch. Elementos con id: titulo
(h1), subtitulo (p), version (span), controles (div vacío), contador (p) y lista (ul).
js/config.js tiene exactamente tres constantes, una por línea: APP_TITULO = "PHP Words",
APP_SUBTITULO = "Palabras reservadas de PHP" y APP_VERSION = "0.1.0".
js/palabras.js declara una única constante global PALABRAS, array de objetos con palabra,
categoria y descripcion. Un objeto por línea; categorías control, funciones, clases,
excepciones y otros; mínimo 25 palabras.
js/app.js define let filtros = { texto: "", categoria: "todas" }; y aplicarFiltros(), que
muestra en #lista las palabras que cumplen los filtros y escribe en #contador "Mostrando N de M
palabras". Al cargar, rellena título, subtítulo y versión desde config.js y llama a
aplicarFiltros().
Las funciones nuevas se entregan siempre como un archivo nuevo en js/ y nunca modifican
archivos existentes. Solo usan filtros, aplicarFiltros(), PALABRAS y #controles. La línea
script que las engancha la añado a mano.
Todo debe funcionar abriendo index.html con doble clic. No hay servidor remoto ni dependencias
externas.
```

Comprobaciones y estructura

- `config.js` contiene tres líneas de declaración.
- El array tiene 30 objetos iniciales, cada uno en una sola línea.
- El HTML carga config, palabras y app en ese orden.
- El README contiene título, descripción y “Pendiente de documentar.”

```
php-words/
├── index.html
├── README.md
├── notas.txt
├── css/estilos.css
└── js/
    ├── config.js
    ├── palabras.js
    └── app.js
```

Captura 1.1 · Prompt enviado a la IA.

PENDIENTE: insertar captura real a pantalla completa.

Captura 1.2 · Aplicación en el navegador con versión 0.1.0 y todas las palabras iniciales.

PENDIENTE: insertar captura real con la fecha/hora y ruta visibles.

Captura 1.3 · Editor con el árbol inicial de archivos.

PENDIENTE: insertar captura real a pantalla completa.

Misión 2 · Mirar antes de guardar

El repositorio local se inicializó con `main`. Se prepararon solo los archivos de aplicación; `notas.txt` quedó sin seguimiento. El primer commit es `3101b0c`.

Comandos de Git

```
git init -b main
git status --short
git add index.html README.md css js
git status --short
git commit -m "Estructura inicial de la aplicación PHP Words"
git --no-pager log --oneline
```

Observa

Antes de preparar, los archivos de la aplicación aparecían como no seguidos (sin seguimiento): Git todavía no los controla. Tras `git add`, los seis elementos seleccionados aparecen como añadidos en staging (A, normalmente verde); `notas.txt` continúa con `??`, es decir, sin seguimiento. Después del commit, esos seis archivos forman el primer registro.

Concepto · Tres espacios

```
Directorio de trabajo       Zona de preparación       Repositorio (.git)
notas.txt (sin seguimiento)  vacía tras el commit        3101b0c:
                            tras preparar:                index.html
                            index.html                    README.md
                            README.md                     css/estilos.css
                            css/                          js/config.js
                            js/                           js/palabras.js
                                                          js/app.js
```

Captura 2.1 · Estado justo después de crear el repositorio.

PENDIENTE: insertar captura real.

Captura 2.2 · Estado abreviado tras preparar los archivos sin notas.txt.

PENDIENTE: insertar captura real.

Captura 2.3 · Historial resumido tras el primer commit.

PENDIENTE: insertar captura real.

Misión 3 · Documentación en su propia rama

En la rama `documentacion` se completó README.md. Primero se cambió la instrucción de ejecución y se preparó; después se añadió una segunda línea sin preparar. El estado `MM` muestra modificaciones tanto en staging como en el directorio de trabajo. El commit final es `dfb3a11`.

Comandos de Git

```
git switch -c documentacion
git diff
git add README.md
git status --short
git diff
git diff --cached
git diff HEAD
git add README.md
git commit -m "Documenta ejecucion"
git commit --amend -m "Documenta cómo ejecutar la aplicación"
git show --stat --oneline HEAD
```

Observa y concepto

En la primera comparación, la línea pendiente antigua lleva `-` y la nueva instrucción lleva `+`. En `MM`, la primera M indica que hay cambios preparados respecto al commit; la segunda, que el archivo volvió a cambiar después de prepararlo.

`git diff` compara el directorio de trabajo con staging; `git diff --cached` compara staging con el último commit; `git diff HEAD` compara el directorio con el último commit, por eso combina ambos cambios. Se puede corregir el último commit con `--amend` porque aún no se había compartido; tras compartirlo, reescribirlo puede divergir la historia de otros repositorios.

Captura 3.1 · Diferencias iniciales del README.

PENDIENTE: insertar captura real.

Captura 3.2 · Estado abreviado con MM.

PENDIENTE: insertar captura real.

Capturas 3.3, 3.4 y 3.5 · Una evidencia para cada una de las tres comparaciones.

PENDIENTE: insertar tres capturas reales.

Capturas 3.6 y 3.7 · Corrección del mensaje y resumen del commit.

PENDIENTE: insertar capturas reales.

Misión 4 · Incorporar la documentación

Se volvió a `main` y se fusionó `documentacion`. Antes de fusionar, el cambio solo existía en la rama de documentación. Git hizo un avance rápido (fast-forward): main no tenía commits divergentes, así que su referencia avanzó hasta el commit ya existente, sin crear un commit de fusión nuevo.

Comandos de Git

```
git switch main
git merge documentacion
git --no-pager log --oneline --decorate --graph --all
```

Captura 4.1 · README en main antes de la fusión.

PENDIENTE: insertar captura real.

Captura 4.2 · Resultado fast-forward de la fusión.

PENDIENTE: insertar captura real.

Captura 4.3 · Grafo con las referencias de las ramas.

PENDIENTE: insertar captura real.

Misión 5 · Tres ideas en paralelo

Las ramas `mas-palabras`, `buscador` y `categorias` se crearon partiendo del mismo commit de main, `dfb3a11`. Cada rama se terminó y se volvió a main antes de crear la siguiente.

5.A · mas-palabras

Se añadieron `abstract`, `static` y `const` al array, manteniendo un objeto por línea. Commit: `f266a78`, “Añade más palabras reservadas”. El total final pasa de 30 a 33.

```
git switch -c mas-palabras
git status --short
git add js/palabras.js
git commit -m "Añade más palabras reservadas"
git switch main
```

5.B · buscador

Se creó un archivo nuevo que añade el campo de texto a `#controles`, actualiza `filtros.texto` y llama a `aplicarFiltros()`. Se añadió su script manualmente y se cambió la versión a 0.2.0. Commit: `1158bcb`.

> # Reconstrucción del prompt de buscador; no es transcripción literal del prompt enviado.

```
Genera únicamente js/buscador.js para PHP Words. Crea un campo de texto dentro de #controles;
al escribir, actualiza filtros.texto y llama a aplicarFiltros(). No modifiques otros archivos:
añadiré yo la línea script. Respeta el contrato técnico de la aplicación: index.html carga
config.js, palabras.js y app.js, en ese orden; existen los ids titulo, subtitulo, version,
controles, contador y lista; config.js define APP_TITULO, APP_SUBTITULO y APP_VERSION; PALABRAS
es el array global con objetos de una línea y las cinco categorías; app.js define filtros y
aplicarFiltros(). Sin módulos, dependencias, fetch ni cambios en archivos existentes. Usa solo
filtros, aplicarFiltros(), PALABRAS y #controles. Elige tú un placeholder apropiado y estilos
simples.
```

```
git switch -c buscador main
git status --short
git add index.html js
git status --short
git commit -m "Añade buscador por texto"
git switch main
```

5.C · categorias

Se creó `js/categorias.js` con las categorías obtenidas de `PALABRAS`, opción “todas” y actualización de filtros al cambiar. Se añadió el script a mano y versión 0.3.0. Commit: `1eb2361`.

> # Reconstrucción del prompt de categorías; no es transcripción literal del prompt enviado.

```
Genera únicamente js/categorias.js para PHP Words. Crea dentro de #controles un desplegable con la opción “todas” y las categorías existentes en PALABRAS. Al cambiar la selección, actualiza filtros.categoria y llama a aplicarFiltros(). No modifiques los archivos existentes: añadiré manualmente la línea script. Respeta el contrato técnico: app estática sin módulos, librerías ni fetch; config.js, palabras.js y app.js ya se cargan en ese orden; están definidos filtros, aplicarFiltros(), PALABRAS y #controles. Las palabras tienen las categorías control, funciones, clases, excepciones y otros. Usa solo esos elementos permitidos y deja abiertos el texto visible y los estilos.
```

```
git switch -c categorias main
git status --short
git add index.html js
git commit -m "Añade filtro por categoría"
git switch main
git --no-pager log --oneline --decorate --graph --all
```

Observa

En buscador y categorías, `index.html` y `js/config.js` son modificados; el archivo JavaScript de la nueva función es sin seguimiento hasta añadirlo. Preparar por nombre permite limitar el commit a los cambios de esa tarea y evita incluir por accidente `notas.txt` u otros archivos.

```
main: dfb3a11
├── mas-palabras: f266a78
├── buscador: 1158bcb
└── categorias: 1eb2361
```

Árbol común final: index.html, README.md, notas.txt (sin seguimiento), css/estilos.css y js/{config.js, palabras.js, app.js, buscador.js, categorias.js}.

Capturas 5.1–5.6 · Prompts, estados, pruebas de navegador y grafo de las ramas.

PENDIENTE: insertar las seis capturas reales solicitadas.

Misión 6 · Fusionar, etiquetar y resolver

Comandos de Git

```
git merge buscador
git tag v0.2.0
git tag --list
git merge --no-ff mas-palabras -m "Fusiona mas-palabras"
git --no-pager log --oneline --decorate --graph --all
git merge categorias
git status --short
git --no-pager log --merge --oneline --decorate
git add index.html js/config.js
git status --short
git commit -m "Fusiona categorias y resuelve conflictos"
```

Conflicto y decisión

La fusión de `buscador` fue fast-forward porque main aún estaba en el ancestro común. La fusión de `mas-palabras` necesitó un commit porque main y la rama ya tenían commits distintos desde ese ancestro; no hubo conflicto porque las ramas editaron zonas distintas: una cambió config/HTML y la otra el array de palabras.

La fusión de `categorias` chocó en `index.html` y `js/config.js`: ambas ramas, partiendo del mismo punto, editaron respectivamente la misma zona de scripts y la línea de versión. Se conservaron ambos scripts en HTML y se eligió 0.3.0, que corresponde a la integración de categorías. `palabras.js` no entró en conflicto porque la rama categorias no modificó ese archivo.

```
Resolución final:
  <script src="js/config.js"></script>
  <script src="js/palabras.js"></script>
  <script src="js/app.js"></script>
  <script src="js/buscador.js"></script>
  <script src="js/categorias.js"></script>
const APP_VERSION = "0.3.0";
```

Verificación funcional

En navegador, la aplicación mostró “Versión 0.3.0” y “Mostrando 33 de 33 palabras”. Al combinar búsqueda “for” y categoría “control”, mostró “Mostrando 2 de 33 palabras” (for y foreach).

Capturas 6.1–6.9 · Fusiones, conflictos, resolución, prueba final y grafo.

PENDIENTE: insertar las nueve capturas reales solicitadas.

Misión 7 · Etiquetas y limpieza

Comandos de Git

```
git tag -a v0.3.0 -m "Versión 0.3.0: buscador, categorías y más palabras"
git tag v0.1.0 3101b0c
git tag -n1
git show --no-patch --format=fuller v0.3.0
git branch -d documentacion mas-palabras buscador categorias
git branch --list
git status --short
git --no-pager log --oneline --decorate --graph --all
```

Observa

Una etiqueta anotada guarda metadatos del etiquetador, fecha, mensaje y objeto etiquetado; una etiqueta ligera es solo una referencia al commit. `v0.1.0` apunta al commit inicial `3101b0c`; `v0.2.0` al commit de buscador `1158bcb`; `v0.3.0` al commit de fusión final `6b84e90`.

`notas.txt` sigue sin seguimiento. Es correcto porque el guion lo define como bloc personal que no forma parte de la aplicación y pide expresamente no añadirlo al repositorio.

Estado final

Elemento	Estado
Rama actual	main
Ramas ya fusionadas	Eliminadas de forma segura; solo queda main
Etiquetas	v0.1.0, v0.2.0 y v0.3.0
Archivos de aplicación	Commit final; estado de trabajo limpio
notas.txt	??, sin seguimiento (intencionado)

```
main (HEAD, v0.3.0)
*   6b84e90 Fusiona categorias y resuelve conflictos
|\
| * 1eb2361 Añade filtro por categoría
* |   50b589f Fusiona mas-palabras
|\ \
| * | f266a78 Añade más palabras reservadas
* | 1158bcb (v0.2.0) Añade buscador por texto
|/
* dfb3a11 Documenta cómo ejecutar la aplicación
* 3101b0c (v0.1.0) Estructura inicial de la aplicación PHP Words
```

Capturas 7.1–7.7 · Etiquetas, borrado seguro de ramas, estado, grafo y aplicación final.

PENDIENTE: insertar las siete capturas reales solicitadas.

Incidencia de preparación del repositorio

Durante la preparación, `git init -b main` se ejecutó inicialmente desde la carpeta superior y el primer `git add` no encontró los archivos porque estaban dentro de `php-words`. Se corrigió moviendo la carpeta `.git` al proyecto. Después, el trabajo siguió en el repositorio correcto `php-words`. Se deja constancia para no presentar esa inicialización como si hubiera ocurrido sin incidencia.

```
git init -b main
git status --short
git add index.html README.md css js
mv .git php-words/.git
git -C php-words status --short
git -C php-words add index.html README.md css js
git -C php-words status --short
git -C php-words commit -m "Estructura inicial de la aplicación PHP Words"
git -C php-words --no-pager log --oneline
```

Nota: esta sección conserva el orden de los comandos relevantes que se ejecutaron; varios comandos de inspección se agruparon en una misma línea de terminal durante la realización.

Reflexión final

> # Borrador con datos ficticios: sustituir estas cinco líneas por una reflexión personal antes de entregar.

Lo más difícil fue entender qué compara cada variante de git diff.

También me costó anticipar el conflicto al fusionar ramas paralelas.

El estado y el historial me ayudaron a decidir el siguiente paso.

En un proyecto real escribiría prompts y comandos en una bitácora desde el inicio.

Además revisaría el staging antes de cada commit y comunicaría antes de reescribir historia.

Antes de entregar

- Sustituir los campos personales `#` por los datos reales.
- Reemplazar todos los marcadores de captura por capturas reales que cumplan el guion.
- Reescribir la reflexión en primera persona y según la experiencia real.
- Verificar con el docente si se admite el nombre de archivo con marcadores: `#Apellidos_#Nombre_UT1_PHPWords.pdf`.